Equality in the initializer

A simpler equality interface for a six-rule presentation of iPRA

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a six-rule presentation of full first-order intuitionistic primitive recursive arithmetic in which the equality instruction has only two proof arguments. It carries no term template, no occurrence path, and no extra numerical parameter. The instruction changes either the root of a term or the initializer of one scalar iterator; the appropriate change is read from the two premise conclusions. Zero iteration is the identity on the initializer, whereas one iteration applies an arbitrary fixed numerical template. Thus the same initializer rule supplies root equality composition and all numerical congruence. General congruence adds just four primitive inference nodes to its supplied equality proof.

The report proves exact consequence equivalence with the current native scalar kernel after zero-only normalization, including the iterator-body and substitution arguments required by both interpretation composites. The interpretation domains are whole, equality is actual equality, and both numerical isomorphisms have graph $x = y$. Complete strict Python checkers, reproducible measurements, proof translations, and executable certificates accompany the written proof. This is a specific reduction of the equality interface, not a proof of absolute minimality or a new global compressed-checker record.

Contents

1 Result and comparison standard **2**

2 A deterministic equality rule with no template parameter **2**

3 Zero-canonical scalar syntax and exact equivalence **9**

4 The arithmetic and logical bridge **13**

5 Complete checker and measurement **15**

6 Executable evidence and its limits **16**

7 Further research questions **16**

Inherited foundations and attribution **18**

A Complementary introductions with one local assumption **18**

B Arithmetic strength and both ordinary interpretation composites **23**

C Internal scalar predecessor bootstrap **25**

D From literal-equation induction to full intuitionistic PRA **29**

E Pairing, primitive recursion, and internal numerical composites **34**

F Complete selected checker **36**

1 Result and comparison standard

The proposed calculus $\mathcal{S}$ is stated completely in Section 2. Its arithmetic syntax is a scalar iterator with only zero-counter computation built into syntax. It retains six inference patterns: hypothesis, successor computation, initializer replacement, closure elimination, paired-binder elimination, and one witness-directed introduction.

The change is in equality. A usual one-template rule asks for

$$\frac{s = t \qquad f(s) = r}{f(t) = r}$$

and makes the checker validate and instantiate the supplied template f. Our primitive rule accepts only the initializer position of an iterator, together with its zero-step degeneration. This restriction makes the conclusion computable from the two proved premise formulas. Arbitrary-template transport is a derived rule.

The comparison is pinned to ProveIt commit

`6bf83d8765614b0af51b35747e07fb057d6b2f42`.

The scoped baseline comprises the current `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` README, the relevant filed equality and normalization reports, and all five archives then present in its `docs/incoming` directory. The incoming canonical-recursors report already gives a six-rule successor-seeded presentation. Reducing eight rules to six is therefore not the new result.

The target called iPRA here is *full first-order* intuitionistic logic with equality, one symbol for each finite primitive recursive definition, its defining equations, the successor axioms, and induction for quantifier-free formulas. It is not merely the quantifier-free equational fragment. First-order and quantifier-free presentations are distinguished in, for example, Avigad [2]. The new native calculus assumes no unlisted logical rules; ordinary logical connectives are derived macros.

1.1 Claims and limits

1. **A smaller primitive equality interface.** One instruction, with two proof arguments and no template data, recovers all equality reasoning in arbitrary retained contexts. Its recovery uses no induction, cases, or binder rule.
2. **Exact equivalence.** After normalization every native sequent is preserved in both directions. Both formula composites are proved; the numerical interpretation domains remain whole and equality remains actual.
3. **A controlled software comparison.** The proposed checker and its one-template control use the same syntax, canonical substitution, logical rules, public validation, and execution contract. A separate shared traversal refactoring is measured separately.

No absolute lower bound is proved. Source length is relative to the specified encoding and runtime; a rule count alone does not determine a shortest checker. The supplied modules are transparent implementations, not globally shortest alternatives to the repository's compressed codecs.

2 A deterministic equality rule with no template parameter

This reduction uses an initialized scalar iterator with its zero equation made definitional. It differs from successor-seeded canonicalization: here a zero-counter iterator disappears entirely.

That feature lets a single fixed-position equality rule include root equality composition.

2.1 Canonical terms and the six rules

Canonical terms have the constructors x, 0, $S(t)$ and $I(n, a; u.h)$, with canonical subterms and with the counter n not literally 0. Only u in h is bound. Define the smart constructor

$$\mathcal{I}(n, a; u.h) = \begin{cases} a, & n \equiv 0, \\ I(n, a; u.h), & n \not\equiv 0. \end{cases}$$

The symbol $\equiv$ denotes literal syntactic identity. In this display the arguments are already canonical. For arbitrary raw terms, normalize all three arguments recursively before applying this smart constructor. Thus normalization must test the *normalized* counter. Write $t\langle\sigma\rangle$ for capture-free substitution followed by normalization. This operation is used throughout the calculus.

Native formulas are $s = t$, $Q_y B$, and $D_{x,y} B$. Their ordinary readings are unchanged:

$$Q_y B = B(0) \to \forall y\, B(y), \qquad D_{x,y} B = \exists x (B(x, 0) \to \forall y\, B(x, y)).$$

All displayed instances of a matrix use canonical substitution. Contexts are finite reusable lists of native formulas. Put $1 = S0$.

For canonical terms s, t, b, define the partial operation

$$\rho_{s,t}(b) = \begin{cases} t, & b \equiv s, \\ I(n, t; u.h), & b \equiv I(n, s; u.h), \\ \text{undefined}, & \text{otherwise.} \end{cases} \qquad \text{(Seed replacement)}$$

The second branch inspects one literal constructor and its initializer; it leaves the counter and update unchanged. The first and second branches are disjoint. If both matched, the finite term s would equal a term having s as a proper initializer subterm, which contradicts strict increase of syntax-tree size. The output of the second branch remains canonical because its counter is the same nonzero literal counter. No bound variable is substituted or renamed by this operation.

Define

$$\eta_{B,t}(z) = \begin{cases} z, & B \text{ has equation root and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

The complete six-rule presentation, denoted $\mathcal{S}$, is:

$$\frac{A \in \Gamma}{\Gamma \vdash A} \qquad \text{Hyp,}$$

$$\frac{}{\Gamma \vdash I(Sn, a; u.h) = h\langle \mathcal{I}(n, a; u.h)/u \rangle} \qquad \text{IS,}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash \rho_{s,t}(b) = r} \quad (\rho_{s,t}(b) \text{ defined}) \qquad \text{Seed,}$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \text{QE,}$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A} \qquad \text{DE,}$$

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)} \qquad \text{DI.}$$

In DE, b is fresh for the retained context, conclusion, and passive matrix parameters. In DI, z is fresh for the retained context, witness t, and passive matrix parameters. Local assumptions are discharged. The designated contradiction is $0 = S0$.

The seed rule has no numerical side parameter in a proof certificate: its two premise conclusions determine the conclusion, or determine rejection. In particular it has neither an arbitrary numerical template nor a user-supplied occurrence path. This does *not* remove substitution from the whole proof checker: IS and the binder rules still require canonical, capture-free substitution.

Lemma 2.1 (Equivalent displayed seed schema)**.** *The deterministic Seed rule is exactly the schema*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash \mathcal{I}(n, s; u.h) = r}{\Gamma \vdash \mathcal{I}(n, t; u.h) = r}, \tag{K}$$

where n, h are arbitrary canonical data. They need not be supplied in certificates.

Proof. At $n = 0$, the schema is the first branch of (Seed replacement); the erased update is irrelevant, and may be chosen as 0. At any other canonical counter, the second premise displays exactly the iterator node from which its counter and update are read. The branches cannot overlap by the finite-size argument above. Conversely, the first deterministic branch is the schema with $n = 0, h = 0$, and the second is its instance with the counter and update displayed by b. □

Lemma 2.2 (Seed replacement commutes with canonical substitution)**.** *If $\rho_{s,t}(b)$ is defined, then for every admissible numerical substitution σ,*

$$\rho_{s\langle\sigma\rangle, t\langle\sigma\rangle}(b\langle\sigma\rangle) = \rho_{s,t}(b)\langle\sigma\rangle.$$

In particular the substituted partial operation is defined.

Proof. In the root branch this is literal equality. In the iterator branch, canonical substitution changes $I(n, s; u.h)$ to the smart constructor with substituted counter, initializer and update. If the substituted counter normalizes to 0, both the old minor and old conclusion lose their iterator nodes, and the new rule uses its root branch. Otherwise both retain their iterator nodes, and the new rule uses its initializer branch. The counter and update are unchanged between the compared expressions in either case. Canonical substitution coherence supplies the stated identity. Thus the equality rule itself is stable even when normalization changes its operational branch. □

2.2 Complete equality reasoning from the seed rule

Every argument in this subsection works under an arbitrary retained context, including arbitrary equation hypotheses. It uses only IS and Seed, apart from using the supplied premise derivations. It uses no induction, numerical cases, or logical binder rule.

The instance of (K) at $n = 0$ is

$$\frac{s = t \qquad s = r}{t = r}. \tag{K0}$$

It is available for *all* source terms, including literal 0. No source restriction is part of this presentation.

Lemma 2.3 (Root equality algebra)**.** *Reflexivity, symmetry and transitivity are derivable.*

Proof. Put $L(a) = I(1, a; u.u)$. IS at predecessor zero gives $L(a) = a$, because the right-hand substituted term is $\mathcal{I}(0, a; u.u) = a$. Using two copies of this axiom as the premises of K0 gives $a = a$.

For symmetry of a supplied $s = t$, use it as K0's major and $s = s$ as its minor; the result is $t = s$. For transitivity of $s = t$ and $t = r$, first obtain $t = s$ by symmetry. K0 with major $t = s$ and minor $t = r$ gives $s = r$. □

Lemma 2.4 (One-step application)**.** *For every fixed canonical numerical template $f(u)$ and ambient term a, IS derives*

$$I(1, a; u.f(u)) = f\langle a/u\rangle. \tag{Beta1}$$

Proof. IS with predecessor zero has right side $f\langle \mathcal{I}(0, a; u.f)/u\rangle$, which is literally $f\langle a/u\rangle$. No equality substitution into f has been used. This is the point at which definitional zero computation is essential. □

Theorem 2.5 (Arbitrary numerical congruence)**.** *For every fixed numerical template $f(u)$, the rule*

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash f\langle s/u\rangle = f\langle t/u\rangle}$$

is admissible in $\mathcal{S}$.

Proof. Abbreviate $f\langle s/u\rangle$ by $f(s)$ and similarly for t, and put $A_a = I(1, a; u.f)$. The entire construction is

$$\dfrac{\dfrac{s = t \qquad A_s = f(s)}{A_t = f(s)}\,\mathrm{Seed}_1 \qquad A_t = f(t)}{f(s) = f(t)}\,\mathrm{Seed}_0.$$

The two endpoint equations $A_s = f(s)$ and $A_t = f(t)$ are IS axioms from Lemma 2.4. The last inference is K0. In particular this construction does not even use the derived symmetry, transitivity, or reflexivity lemmas. The template may contain zero, one, or several occurrences of its designated variable, including occurrences under nested iterator binders. All such occurrences are handled by IS's ordinary capture-free substitution, not by the primitive equality rule. □

Proposition 2.6 (Four-node congruence expansion)**.** *If the supplied derivation of $s = t$ has P inference nodes, the displayed derivation of $f(s) = f(t)$ has exactly $P + 4$ nodes: two IS axioms and two Seed applications are added, and the supplied premise is inserted once. The bound is independent of the syntactic depth and number of occurrences of the distinguished variable in f.*

Proof. Count the four displayed additions in the preceding proof tree. □

This is a count of the specified inference tree. Its term labels contain copies of f, s, and t; it is not a constant bound on serialized certificate size, runtime, or the shortest possible proof. The prior wrapped counter-congruence rule in `witness_rule/theory/restricted_congruence.tex` has a specified $P + 5$ construction using one counter-congruence inference, two endpoint computations, and two separate root replacements. The present rule fuses the lifting and the first root replacement at the inference interface; the zero-counter instance of that same rule supplies root replacement. The numerical language and its normalization convention differ, so this comparison should not be read as a uniform shortening of every old proof.

Corollary 2.7 (One-template transport)**.** *The admissible rule*

$$\frac{s = t \qquad f(s) = r}{f(t) = r}$$

holds with arbitrary source terms and arbitrary contexts.

Proof. Theorem 2.5 supplies $f(s) = f(t)$. K0 with this major and the supplied minor gives the result. □

Corollary 2.8 (One-template crossed transport)**.** *The admissible rule*

$$\frac{s = t \qquad f(s) = r}{r = f(t)}$$

holds with arbitrary source terms and arbitrary contexts.

Proof. Corollary 2.7 gives $f(t) = r$; apply derived symmetry. □

Corollary 2.9 (Full equational transport)**.** *For arbitrary numerical templates $a(u), b(u)$,*

$$\frac{s = t \qquad a(s) = b(s)}{a(t) = b(t)}$$

is admissible.

Proof. Apply Corollary 2.8 first with $f = a, r = b(s)$, obtaining $b(s) = a(t)$, and then with $f = b, r = a(t)$, obtaining $a(t) = b(t)$. These are two uses of an admissible rule, not two primitive seed instructions; their numerical-congruence expansions contain computation and root-composition steps. □

Weakening, fresh renaming, and ordinary-assumption cut in $\mathcal{S}$ follow directly by induction on derivations. They leave all numerical expressions at a rule instance unchanged, up to injective renaming. Thus neither the DI mode nor the deterministic Seed branch changes. General numerical substitution is addressed after the comparison theorem; it should not be claimed by a direct induction on restricted DI.

2.3 Recovering both ordinary introductions

Lemma 2.10 (A canonical nonliteral zero)**.** *The term*

$$Z := I(1, 0; u.0)$$

is canonical, is not literally 0*, and has an IS proof of $Z = 0$.*

Proof. Its counter is $S0$, so canonicalization does not erase its outer node. IS with predecessor zero and constant-zero update has right side 0. □

The old zero-counter alias $I(0, 0; u.0)$ cannot serve here: it is literally 0 in the canonical syntax. The one-step alias in Lemma 2.10 is the required replacement.

Lemma 2.11 (Unrestricted case introduction)**.** *For every matrix B, every witness t, and the usual fresh z,*

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)} \qquad \text{(D-case)}$$

is admissible in $\mathcal{S}$.

Proof. The displayed instance already is primitive DI unless B is an equation and t is literally zero. In the exceptional case write $B = E$ and let the supplied derivation be

$$\pi : \quad \Gamma, E(0, 0) \vdash E(0, Sz).$$

Choose Z from Lemma 2.10. Under $\Gamma, E(Z, 0)$, transport its local hypothesis along $Z = 0$ using Corollary 2.9, obtaining $E(0, 0)$. Ordinary-assumption cut with π gives $E(0, Sz)$. Reverse-transport along $0 = Z$ to obtain $E(Z, Sz)$. DI at the nonliteral witness Z is in its case mode and concludes the desired $D_{x,y}E(x, y)$.

Only equation transport is used, since E has equation root. No transport through a binder, numerical substitution into π, or substitution-admissibility theorem for the restricted calculus is assumed. □

Lemma 2.12 (Direct equation progress)**.** *For every literal equation $E(y)$,*

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)} \qquad \text{(Q-progress)}$$

is admissible in $\mathcal{S}$.

Proof. Choose a witness variable x absent from E and apply primitive DI to $D_{x,y}E(y)$ at witness 0. Its mode is progress, and its premise is precisely the given one. DE opens this formula with a fresh witness; the resulting local assumption is $Q_y E(y)$, independent of that witness. Hyp proves it in the opened context and DE exports it. □

Lemma 2.13 (Conditional closure introduction)**.** *For arbitrary B, D-case and DE derive*

$$\frac{\Gamma, B(0) \vdash B(Sz)}{\Gamma \vdash Q_y B(y)}. \qquad \text{(Q-case)}$$

Proof. Use D-case at an unused witness coordinate and then DE plus Hyp, exactly as in Lemma 2.12. □

2.4 The stable auxiliary calculus and formula transport

Let $\mathcal{A}$ use the same canonical terms, formulas, and contexts. Its rules are Hyp, IS, unrestricted equational transport, QE, DE, D-case, and Q-progress. This is the canonical counterpart of the native kernel with zero computation made definitional.

Theorem 2.14 (Exact sequent equivalence)**.** *For every finite context Γ and native formula A,*

$$\Gamma \vdash_{\mathcal{S}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{A}} A.$$

Proof. From $\mathcal{A}$ to $\mathcal{S}$, recursively translate premise derivations. Corollary 2.9 replaces equational transport, Lemma 2.11 replaces D-case, and Lemma 2.12 replaces Q-progress. Other rules are unchanged.

Conversely, the standard equality algebra is derivable in $\mathcal{A}$ from IS and unrestricted equational transport, starting again with the computation $L(a) = a$. An instance of K follows by transport along $s = t$ inside the single numerical template $\mathcal{I}(n, v; u.h)$, keeping n, h fixed. This also covers $n = 0$, when that canonical template is just v. Each DI case-mode inference is already D-case.

For a progress-mode DI, the witness is 0 and its matrix is an equation $E(x, y)$. Q-progress applied to the supplied premise gives $Q_y E(0, y)$. Under $E(0, 0)$, QE at Sz yields $E(0, Sz)$. D-case with witness 0 now concludes $D_{x,y}E(x, y)$. This translates every remaining primitive rule and preserves end sequents. □

Theorem 2.15 (Full formula transport)**.** *For every native formula template $F(v)$,*

$$s = t, \;\; F(s) \;\; \vdash_{\mathcal{S}} \;\; F(t).$$

Proof. Use external structural induction on F, uniformly in the retained context and all passive parameters. Equation roots use Corollary 2.9.

Suppose $F(v) = Q_y B(v, y)$. To introduce $Q_y B(t, y)$ by Q-case, assume $B(t, 0)$. Symmetry and the induction hypothesis convert this to $B(s, 0)$. QE for the given $Q_y B(s, y)$ gives $B(s, Sz)$, and the induction hypothesis transports this to $B(t, Sz)$. Discharge the base using Q-case.

Suppose $F(v) = D_{x,y} B(v, x, y)$. DE opens the source at a fresh witness b, supplying $Q_y B(s, b, y)$. Introduce the target by D-case at the same witness. Its local base $B(t, b, 0)$ is transported backwards to $B(s, b, 0)$, QE gives $B(s, b, Sz)$, and forward transport gives $B(t, b, Sz)$. Close D-case and DE. Every recursive call concerns the strictly smaller matrix B; no arithmetic induction on a compound formula has been introduced. □

The auxiliary calculus is stable under canonical numerical substitution: normalization coherence handles IS and all instances, formula roots are preserved, and eigenvariables are freshened. Theorem 2.14 therefore gives substitution admissibility for $\mathcal{S}$ as well. This is a derived admissibility result. A substitution can change a witness into literal 0, so a direct rule-for-rule substitution proof for its six-rule presentation would be invalid.

2.5 Interpretations and the precise novelty claim

Once the raw/canonical comparison is established, the existing native kernel's whole-domain bi-interpretation with iPRA transfers by Theorem 2.14. The numerical domains remain whole, equality remains actual equality, and the numerical composite isomorphisms have graph $x = y$. The raw/canonical comparison must prove normalization correctness under iterator bodies; it cannot replace that obligation with a purely syntactic claim that the ordinary theories are identical. Literal equation induction supplies the needed body extensionality. Section 3 proves that comparison and its full formula composites.

The existing sources already contain several ingredients:

- `Research/counter_congruence/theory/counter_theorem.tex` derives arbitrary context congruence from root replacement plus counter-position congruence for a *counter-aware*, successor-seeded recursor.
- `Research/witness_rule/theory/restricted_congruence.tex` further gives a wrapped, state-independent counter rule, a separate root rule, and a five-node congruence expansion. It explicitly factors arbitrary templates through a fixed computation shape.
- `Research/one_context` and the incoming `ipra_one_sided_transport_2026-11-11.zip` use one-template crossed transport. The single witness-directed introduction is inherited from `witness_rule` and `NativeOneIntro.lean`.
- The incoming `ipra_canonical_recursors_2026-11-12.zip` makes the zero computation of a successor-seeded recursor canonical by erasing its update body. It does not erase the whole recursor node and does not make its zero step the identity function of the initializer.

The present specific change is that *one initialized scalar seed rule*, with zero computation made definitional, supplies both root composition (counter 0) and arbitrary context congruence (counter 1). Its conclusion is determined by its two premise conclusions. No template, term, or occurrence path need be supplied to the equality instruction. This is a mathematical reduction of the equality interface, followed by an implementation opportunity. It is not a claim that fixed-position congruence is a new idea, that normalization has no trusted cost, or that an absolute smallest iPRA axiomatization has been found.

All source paths above refer to the commit stated in Section 1, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

3 Zero-canonical scalar syntax and exact equivalence

Reference calculus. Let N be the pinned upstream native scalar calculus with raw terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

formulas $(s = t)$, $Q_y B$, and $D_{x,y} B$, and the eight inference patterns Hyp, I_0, I_S, equational transport, QE, DE, DI by cases, and QI by equational progress. In particular,

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

We use the upstream admissibility of weakening, cut, capture-free numerical substitution, equality algebra, and transport in compound formulas. None of the new normalization claims is needed to establish those upstream facts. All terms and bodies below are fixed finite syntax; no numerical variable ranges over descriptions of functions.

3.1 A normalizer which only contracts zero counters

For already normalized n, a, h, define the smart constructor

$$C(n, a; u.h) := \begin{cases} a, & n \text{ is literally } 0, \\ I(n, a; u.h), & \text{otherwise.} \end{cases}$$

Define ν on raw syntax by

$$\nu x = x, \quad \nu 0 = 0, \quad \nu(St) = S\nu t, \quad \nu I(n, a; u.h) = C(\nu n, \nu a; u.\nu h).$$

A canonical term is a fixed point of ν. It contains no iterator with literal zero counter. The bound variable belongs only to the body, so returning the initializer when a node contracts never removes a binder from the initializer's scope. Extend ν to formulas and contexts by normalizing numerical terms and preserving every formula constructor.

Important distinction from the incoming canonical-R report. Here ν can change a nonzero outer constructor into literal zero. The test at an iterator must therefore inspect νn, not the original n. For example,

$$I(I(0, 0; v.0), a; u.h) \quad \text{normalizes to} \quad \nu a.$$

The false assertion $\nu t \equiv 0 \iff t \equiv 0$ must not be imported from the successor-seeded setting.

Lemma 3.1 (Elementary properties of normalization)**.** *The map ν is idempotent, commutes with renaming, introduces no new free variables, and is primitive recursive on codes of finite syntax trees. It produces a term no larger than its input tree.*

Proof. Structural recursion performs the displayed construction. A contraction only deletes an iterator node and its counter and body, retaining its already normalized initializer; otherwise the recursively normalized subtrees are retained under the same constructor. Consequently the result has no zero-counter node and has no more nodes than the input. Idempotence, renaming, and the free-variable assertion follow by induction. Structural recursion on finite coded trees, with these size bounds, is a primitive-recursive syntactic operation. This procedure never executes successor-counter iteration: $I(Sn, a; h)$ is retained, even for a closed standard numeral n. Thus no evaluator for arbitrary primitive-recursive programs is being treated as a primitive-recursive operation. □

For a capture-free simultaneous substitution σ, write

$$T\langle\sigma\rangle := \nu(T[\sigma]).$$

Under a binder, substitution is lifted in the usual capture-free manner (or by de Bruijn lifting). Images may first be normalized without changing the result.

Lemma 3.2 (Substitution coherence)**.** *For every raw term or formula T and every capture-free simultaneous substitution σ,*

$$\nu(T[\sigma]) = \nu((\nu T)[\nu \circ \sigma]).$$

In particular, normalized substitution is associative and has the usual identity and binder-lifting laws on canonical syntax.

Proof. Induct on T, freshening bound names if names rather than indices are used. At an iterator, normalize its substituted counter first. The counter induction hypothesis says that both sides obtain exactly the same normalized counter. If that counter is zero, both return the same normalized substituted initializer. Otherwise both retain an iterator with the same normalized counter, initializer, and body.

There is one apparently exceptional case: the original normalization may already have deleted the iterator because $\nu n = 0$. The counter induction hypothesis then says $\nu(n[\sigma]) = \nu(0[\nu \circ \sigma]) = 0$, so it is deleted after substitution as well. Formula constructors are preserved and their binder cases use the same argument with lifted substitutions. Associativity follows by applying the equation twice together with associativity of ordinary capture-free substitution. □

3.2 Body extensionality is a proof obligation

The zero normalizer can simplify an iterator's body. Numerical congruence for an ordinary argument of a function symbol does not by itself justify replacing the entire description of its body. The following explicit lemma supplies what is needed.

Lemma 3.3 (Raw iterator body extensionality)**.** *Suppose u is absent from Γ, n, a and the raw calculus proves*

$$\Gamma \vdash_N h(u) = k(u).$$

Then

$$\Gamma \vdash_N I(n, a; u.h) = I(n, a; u.k).$$

The derivation uses induction for a literal equation only.

Proof. Choose z fresh and put

$$H_z = I(z, a; u.h), \qquad K_z = I(z, a; u.k), \qquad E(z) = (H_z = K_z).$$

The base $E(0)$ follows from two zero computations and equality algebra. In the step, assume $E(z)$. Substituting H_z for u in the supplied proof gives $h(H_z) = k(H_z)$ in the same retained context, since u is absent from Γ. Congruence for k applied to $H_z = K_z$ gives $k(H_z) = k(K_z)$. The two successor computations then give the equality path

$$I(Sz, a; h) = h(H_z) = k(H_z) = k(K_z) = I(Sz, a; k).$$

QI by equational progress supplies $Q_z E(z)$; QE with the base proves $E(n)$. No induction on a quantified formula is used. □

Lemma 3.4 (Normalization is provably sound)**.** *For each raw numerical term t, with all its free parameters retained,*

$$\vdash_N t = \nu t.$$

For each raw native formula A, the formulas A and νA are interderivable in every retained context. In the ordinary first-order realization this is the provable biconditional $A \leftrightarrow \nu A$.

Proof. Use external induction on finite term syntax. Variables and zero use reflexivity; successor uses numerical congruence. At $I(n, a; u.h)$, first use the counter and initializer induction hypotheses to replace n, a by $\nu n, \nu a$. The body induction hypothesis, with u still a free numerical parameter, is $h(u) = \nu h(u)$. Lemma 3.3 replaces the body. We have reached $I(\nu n, \nu a; u.\nu h)$; if $\nu n = 0$, one zero computation reaches νa, and otherwise this is already νt.

For the formula assertion, equality atoms use the proved term equalities and equality algebra. It remains to lift pointwise interderivability through the native binders; this must not be confused with capturing a bound variable in a single equality-transport template.

Suppose $B(y)$ and $B'(y)$ are pointwise interderivable. Under $Q_y B$, assume $B'(0)$, translate to $B(0)$, apply QE at Sz, and translate $B(Sz)$ to $B'(Sz)$. Derived case-introduction for Q gives $Q_y B'$. This derived rule follows from DI by cases with an unused existential coordinate, followed by DE whose local goal is its retained Q clause. Reverse the pointwise translations for the converse implication.

For $D_{x,y}B$, apply DE at a fresh b, retaining $Q_y B(b, y)$. To introduce $D_{x,y}B'$ at witness b, assume $B'(b, 0)$, translate to $B(b, 0)$, use QE at Sz, and translate to $B'(b, Sz)$. DI by cases and then DE finish the derivation. Reverse the translations for the other direction. Applying these two lifting constructions in an external induction on formula formation proves the assertion. No arithmetic induction on a quantified formula is added. □

3.3 An unrestricted canonical comparison calculus

Define C_* on canonical syntax using precisely these eight patterns: Hyp, reflexivity, successor computation, equational transport, QE, DE, DI by cases, and QI by equational progress. Every substitution in these rules is normalized. Successor computation is

$$C(Sn, a; u.h) = h\langle C(n, a; u.h)/u\rangle.$$

Equational transport has an arbitrary literal-equation template $E(v)$: from $s = t$ and $E\langle s/v\rangle$, infer $E\langle t/v\rangle$. There is no source restriction and the two introduction modes are separate. Thus this intermediate calculus is stable under canonical substitution and does not yet involve the literal-zero witness test.

Theorem 3.5 (Raw/canonical equivalence)**.** *For every raw finite context Γ and raw native formula A,*

$$\Gamma \vdash_N A \quad \Longleftrightarrow \quad \nu\Gamma \vdash_{C_*} \nu A.$$

Both directions are effective transformations of finite derivations.

Proof. For the forward direction, normalize every label of the raw derivation. A zero computation becomes reflexivity. A successor computation becomes the displayed canonical successor computation by Lemma 3.2. Equational transport, QE, DI by cases, and QI by progress are instances of their canonical versions by the same coherence lemma. Hyp and DE retain their shapes. Freshness is preserved because normalization introduces no free variables. The normalized matrix of an equation is still an equation.

For the reverse direction, view canonical labels as raw syntax. A normalized instance $B\langle\sigma\rangle$ is interderivable in N with the corresponding ordinary instance $B[\sigma]$ by Lemma 3.4. Consequently

each canonical logical or equational inference can be translated by undoing normalization on its premises, applying the raw rule, and normalizing its conclusion. Successor computation translates using the raw successor equation, $I(n, a; h) = C(n, a; h)$ (the zero equation if $n = 0$, reflexivity otherwise), and normalization soundness on the substituted body. Reflexivity is already derived in N. Cut composes these finite derivations. This yields $\nu\Gamma \vdash_N \nu A$; use normalization soundness on each assumption and the conclusion to obtain $\Gamma \vdash_N A$. □

For a canonical context and goal, the theorem says exactly $\Gamma \vdash_{C_*} A$ iff $\Gamma \vdash_N A$. The round trip from canonical syntax through raw syntax is literally the identity. The other round trip is ν and is internally the identity by Lemma 3.4.

3.4 Interface to the six-rule seed calculus

Section 2 supplies the equality and introduction reductions from C_* to its six-rule calculus $K = \mathcal{S}$. The auxiliary calculus $\mathcal{A}$ there has seven rules; C_* additionally lists reflexivity, which is already derived from IS and equational transport. Thus $C_* = \mathcal{A}$ on every canonical sequent. The required seed rule is

$$\frac{s = t \qquad C(n, s; u.h) = r}{C(n, t; u.h) = r}.$$

It has no source restriction. At $n = 0$ it becomes root replacement $s = t,\ s = r \implies t = r$. With $c_s : I(S0, s; u.u) = s$, two copies of c_s in this root instance give $s = s$. A major $s = t$ and minor $s = s$ then give symmetry $t = s$. A major $t = s$ and minor $t = r$ give transitivity $s = r$.

At $n = 1$, write $A_s = I(S0, s; u.h)$ and $A_t = I(S0, t; u.h)$. Successor computation gives $A_s = h\langle s/u\rangle$ and $A_t = h\langle t/u\rangle$. Seed transport along $s = t$ gives $A_t = h\langle s/u\rangle$. Now use this last equation as the major, the second computation as the minor, and the root instance of Seed. The result is exactly

$$h\langle s/u\rangle = h\langle t/u\rangle.$$

Thus two successor computations and two Seed inferences give every term-template congruence. From a supplied P-node proof of $s = t$ the construction has $P + 4$ primitive-inference nodes, counting repeated subproofs separately. This is not a bound on certificate bytes or runtime: the labels still contain the chosen template and endpoints. Equality algebra and congruence recover full equational transport. In the other direction the seed rule is directly a special case of equational transport in C_*.

The one-introduction rule is the usual witness fold: use progress exactly when the matrix is an equation and the canonical witness is literally zero, and cases otherwise. The closed canonical term

$$d := I(S0, 0; u.u)$$

is not literal zero and has the one-step computation $d = 0$. Therefore the omitted equation-case instance with zero witness is recovered by transporting its hypothesis and conclusion between witnesses 0 and d, then applying the permitted case instance at d. Only atomic transport is required, since only an equation matrix loses that case. The unneeded existential coordinate of a progress instance is eliminated to recover QI by progress. These reductions establish $K = C_*$ on all canonical sequents.

The literal-zero witness test is not preserved by direct substitution into the same inference labels. Do not prove substitution admissibility by silently assuming that it is. Establish the unrestricted equivalence above first, then transfer admissibility through $K = C_*$. Weakening and assumption cut themselves preserve labels and witness tests and have the usual direct proofs after freshening eigenparameters.

Template-free checking of seed transport. If the major equation is $s = t$ and the minor is $b = r$, its conclusion can be checked without supplying n, h, or any equality template in the certificate. If b is literally s, output $t = r$. Otherwise require $b = I(n, s; u.h)$ and output $I(n, t; u.h) = r$. The latter counter is already nonzero by canonical syntax. These alternatives implement exactly the smart-constructor rule. A finite acyclic term cannot both equal s and contain s as its immediate initializer, so the alternatives do not conflict. This is a restriction of the equality inference interface, not omission of a template validator or a free appeal to arbitrary congruence.

3.5 Whole-domain bi-interpretability transfer

The pinned upstream result supplies ordinary whole-domain interpretations $F : N \to \mathsf{iPRA}$ and $G : \mathsf{iPRA} \to N$, actual equality, and both internal composite isomorphisms with graph $x = y$. Let $e : C_* \to N$ be the raw reading of canonical syntax and $q : N \to C_*$ be normalization. The preceding theorem and its term and formula lemmas prove

$$qe = \mathrm{id} \quad \text{literally on canonical syntax,} \qquad eq \simeq \mathrm{id} \quad \text{internally in } N.$$

Define the interpretations for $K = C_*$ by

$$F_K := F \circ e, \qquad G_K := q \circ G.$$

Both domains are still the entire numerical domain. Normal forms are representatives of *expressions*, not a restriction or quotient of the domain of numbers. Equality is actual equality throughout.

For a canonical term or formula, the composite is $qGFe \simeq qe = \mathrm{id}$ by the upstream round trip and normalization. In the other direction it is $FeqG \simeq FG \simeq \mathrm{id}$. The analogous assertions for term equalities and arbitrary formula biconditionals follow from the corresponding upstream assertions and Lemma 3.4. Hence both numerical composite graphs remain $x = y$. In particular K and full first-order iPRA are bi-interpretable and equiconsistent. The designated contradiction $0 = S0$ is unchanged by normalization, so the exact derivability equivalence also transfers inconsistency directly.

Ordinary first-order reading. As upstream, binder notation for I abbreviates a finite dependency ordered family of first-order function descriptions. The canonical notation imposes the equation $I(0, a; h) = a$ as a definitional spelling convention and uses normalized substitution. Expanding this convention gives its ordinary first-order realization. Lemmas 3.2 and 3.4 are precisely what makes the expansion commute with substitution up to provable equality. The argument does not identify a normalization algorithm with an internal arithmetic evaluation function and does not postulate a function sort or a uniform evaluator.

Scope of the claim. This establishes a genuine finite inference-interface reduction and a whole-domain bi-interpretation. It does not prove absolute minimality, does not establish any checker byte improvement without actual complete implementations and a matched comparison, and is a written proof rather than a new Lean or Rocq development.

4 The arithmetic and logical bridge

The new work is the initializer rule and its exact comparison proof. The arithmetic bridge is inherited from the pinned ConciseIPRA development [1]. Its five relevant written developments are reproduced with attribution in Appendices A–E. Their exact Git blob identities and SHA-256 hashes are recorded in `evidence/source_provenance.json`.

4.1 Ordinary realization

Expand each fixed iterator body into a first-order function description. No variable ranges over programs. The ordinary theory T_I has intuitionistic first-order equality logic, the two raw iterator equations, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

the separation bridge $0 = S0 \to \bot$, and contextual induction for literal equations. The native readings are

$$(Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (D_{x,y} B)^\circ = \exists x(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)).$$

The separation bridge must be stated in this ordinary realization: positive equations alone have a singleton model. In the native presentation, contradiction is designated as $0 = S0$ and explosion is derived.

4.2 Recovering all ordinary connectives

Put $C(v, a, b) = \mathcal{I}(v, a; u.b)$, with u unused. This selects a at zero and b at successors. External recursion on finite formulas gives a gate $G_v A$, provable at zero and interderivable with A at successors, and a selector $M_v(A, B)$ choosing A at zero and B at successors. At equation roots,

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad M_v(s = t, a = b) = (C(v, s, a) = C(v, t, b)).$$

They extend through the paired binder by padding and congruence (Appendix A). An auxiliary $Q_y B$ is interderivable with $D_{w,y} B$ for an unused w, so these constructions cover the full Eq/Q/D grammar.

For example,

$$A \Rightarrow_* B = D_{w,y} M_y(A, B), \qquad \exists_* x\, A(x) = D_{x,y} G_y A(x), \qquad A \vee_* B = \exists_* v\, M_v(A, B).$$

Conjunction uses a gate and a two-state parity iterator so that two instances extract its conjuncts. Universal quantification combines the zero instance with successor instances supplied by a paired closure. The appendix states every macro and proves its introduction, elimination, and round-trip rules. No induction for compound formulas is added.

4.3 Compiling primitive recursion and proving the composites

The inherited internal construction first obtains a scalar predecessor using equation induction and numerical cases. It then obtains a numerical reflection of actual equality, recovers quantifier-free induction, and constructs pairing and projections. This dependency order avoids assuming pairing or an equality test in the predecessor bootstrap.

With derived pairing π and projections L, H, a recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is represented by

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The iterator binds only its state u; the counter is stored in the paired state. Equation induction proves $L(V(n, \vec{p})) = n$. This is an external compiler for each fixed finite PR description.

Conversely, each fixed scalar iterator is a primitive recursion whose update ignores the recursion counter. Internal uniqueness by equation induction proves the numerical round trips for both

compilers. Section 3 proves the additional canonical-syntax round trip. The logical macros supply both formula composites. Hence both numerical composite maps are the identity with graph $x = y$, and the ordinary realization is bi-interpretable and equiconsistent with full first-order iPRA.

5 Complete checker and measurement

5.1 Public contract

Input is exactly a JSON object with fields `goal` and `proof`. The goal is a closed native formula and the proof is a finite tree of six tagged instructions. Terms use exact integers for zero and de Bruijn indices, one-element lists for successor, and three-element lists for iterators. Only the iterator body has an additional variable in scope. Formula tags distinguish equality, Q, and D, binding zero, one, and two variables respectively.

The checker validates the public goal, every consumed certificate field, scopes, instruction arities, and premise and conclusion matches. Canonicality rejects a literal zero counter in a stored iterator. Smart substitution erases such a node when substitution exposes one. Exact integer checks reject booleans in numerical positions. Acceptance checks do not use `assert`, so optimized Python execution preserves the intended contract.

Acceptance is exactly `ACCEPT` with exit status zero. Invalid inputs print `REJECT` and return a nonzero status. The mathematical certificate system contains all finite proof trees; an executable implementation is subject to ordinary finite resources. The Python standard library and JSON decoder are part of the stated execution environment.

5.2 Premise invariants and the equality instruction

Induction on successful calls to `derive` proves that its result is a canonical native formula in the current scope. A native formula has list arity three exactly when it is an equation; Q and D formulas have arity two. Both matched implementations therefore unpack an equation premise into three fields: a non-equation premise fails and is rejected by the public wrapper.

For proved equations $s = t$ and $b = r$, Seed checks whether $b = s$. If so it returns $t = r$. Otherwise b must be an iterator $I(n, s; u.h)$, and the result is $I(n, t; u.h) = r$. The retained counter remains nonzero and the retained body keeps its scope. No equality-template substitution is performed.

The selected version can reject impossible integer and successor shapes by an exception caught by the wrapper. This uses the proved well-formed-output invariant; it never grants permission to index an arbitrary unvalidated user term. An explicit-shape version is included for comparison. The matched control uses the same premise invariant and public boundary.

5.3 Whole-module measurements

Complete module	Tokens	AST nodes	Bytes	Gzip 9
One-template control	1202	1360	4029	1173
Initializer rule (selected)	1192	1347	3979	1175
Initializer rule (explicit guards)	1236	1404	4156	1226
Canonical R, inherited	1338	1485	4611	1205
Canonical R, shared traversal	1224	1378	4079	1188

The matched initializer change saves 50 normalized bytes (1.2410%), 10 significant tokens, and 13 AST nodes. Gzip-9 grows by 2 bytes.

Measurement removes actual Python docstrings, runs `ast.unparse`, and appends one LF. Significant tokens exclude comments, encoding and end markers, indentation, dedentation, and newline tokens. AST nodes are counted in the normalized module. Gzip uses level 9 and timestamp zero; it is informational. The reference runtime is CPython 3.12.14.

The one-template/initializer pair isolates the mathematical change. The canonical-R pair isolates sharing four binder-aware traversals in an otherwise unchanged calculus. Comparing the proposed I checker with the older R checker combines different grammars and implementation decisions; the whole difference must not be credited to the initializer theorem.

The inspected incoming reports also contain a 516-byte guarded compact profile and an 870-byte fully validated Eq/Q/D profile. Their encodings differ from these transparent modules. This work does not claim a new global byte, token, or AST record.

6 Executable evidence and its limits

- The new calculus and matched controls passed 600 command-line checks across readable and normalized sources, under ordinary and optimized Python, with no mismatch.
- Thirty closed proof-translation round trips preserved their conclusions. An additional open-context round trip kept the counter and body parameters fixed when replacing the initializer.
- The new syntax passed 180 independent numerical substitution/evaluation comparisons and explicit nested-counter, state-capture, and nested-formula capture controls.
- The separate shared traversal refactoring passed 516 process checks on 86 inherited named cases, 480 independent numerical comparisons, and 2400 exact old/new traversal comparisons.

The proof producer translates general one-template instructions into initializer proofs and translates initializer proofs back. It checks the translated conclusions. The fixtures include arithmetic computations, zero-source equality, nested iterator bodies, binder capture controls, logical rules, and invalid proofs. Independent evaluation/substitution comparisons cover bounded numerical examples.

These are executable checks of particular certificates and programs. The universal equivalence and bi-interpretation claims rest on the written arguments above. No new Lean or Rocq proof has been compiled. The upstream README reports a Lean formalization of the inherited bridge; that project was inspected as a source, not rebuilt here.

The delivered reproduction scripts run offline with Python's standard library. Run `python3 reproduce.py` from the archive root to check measurements and rerun the suites. Run `python3 build.py` to assemble the standalone `report.tex` and compile its PDF with pdfLaTeX. The selected checker itself needs no auxiliary files.

7 Further research questions

1. **Formalize the new comparison.** Add zero-canonical scalar terms, normalized substitution, iterator-body extensionality, and Seed to the existing Lean project. First compare the stable calculus with separate case and progress rules; only then transfer to witness-directed DI.
2. **Exploit the missing template field in a compact codec.** Determine whether premise-inferred equality improves the best validated compressed implementations after all

normalization and boundary costs are counted.

3. **Measure proof size and checking time.** The congruence construction adds four inference nodes, but its labels contain copies of the template and endpoints. Measure serialized proofs and runtime on the arithmetic library, distinguishing proof trees from explicitly shared proof DAGs.

4. **Compare other recursor positions.** Which arithmetic primitives support both identity and arbitrary application through one distinguished argument? Require actual equality and both formula composites in every comparison.

5. **Formulate meaningful lower bounds.** A lower-bound problem should fix the term grammar, certificate language, validation contract, and allowed primitive operations. No coding-independent absolute minimum is established here.

Inherited foundations and attribution

The next five sections are included verbatim from the pinned upstream written development. They keep their original local notation: I is the raw scalar iterator with both computation equations, and their $\mathcal{C}, \mathcal{N}$ denote earlier calculi. The proved raw/canonical comparison explains their applicability; they add no primitive rule to the present $\mathcal{S}$.

The MIT No Attribution license is included in `reference/LICENSE-ProveIt.txt`. Historical citation keys `erasure`, `forward`, and `progress` below refer to the corresponding inherited source files at the pinned commit.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and

$\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \qquad \text{(Gate and padding)}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation.

In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [3] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n)+2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [4]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\,E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S((Sx) + y) = S(S(x + y)) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [5]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a+b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a+b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Complete selected checker

The following is the complete selected module, not an excerpt. The standalone file in `code/` is executable directly.

```
"""Template-free transport using proved-term invariants.
Strict six-rule zero-canonical scalar arithmetic; ordinary Q elimination.
The entire command-line checker is contained in this one module.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def rec(n, a, h):
    return a if n == 0 else [n, a, h]


def successor(a):
    return [a]


def term(t, n):
    if type(t) is int:
        return 0 <= t <= n
    if type(t) is not list:
        return False
    if len(t) == 1:
        return term(t[0], n)
    return (len(t) == 3 and t[0] != 0 and term(t[0], n) and term(t[1], n)
            and term(t[2], n + 1))


def formula(a, n):
    if type(a) is not list or not a:
        return False
    if a[0] == '=':
        return len(a) == 3 and term(a[1], n) and term(a[2], n)
    return (len(a) == 2 and a[0] in ('Q', 'D')
            and formula(a[1], n + (1 if a[0] == 'Q' else 2)))


def action(t, f, depth=0):
    if type(t) is int:
        return t if t <= depth else f(t - depth, depth)
    if type(t[0]) is str:
        extra = 0 if t[0] == '=' else 1 if t[0] == 'Q' else 2
        return [t[0]] + [action(a, f, depth + extra) for a in t[1:]]
    if len(t) == 1:
        return [action(t[0], f, depth)]
    return rec(action(t[0], f, depth), action(t[1], f, depth),
               action(t[2], f, depth + 1))


def shift(t, amount=1, depth=0):
    return action(t, lambda i, d: i + d + amount, depth)


def subst(t, images, depth=0):
    return action(t, lambda i, d: shift(images[i - 1], d)
```

```
                if i <= len(images) else i + d - len(images), depth)


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'h' and len(proof) == 2:
        index = proof[1]
        if type(index) is int and 0 <= index < len(context):
            return context[index]
        fail()
    if tag == 'r' and len(proof) == 4:
        n, a, h = proof[1:]
        if not (term(n, scope) and term(a, scope) and term(h, scope + 1)):
            fail()
        return ['=', rec(successor(n), a, h),
                subst(h, [rec(n, a, h)])]
    if tag == 'x' and len(proof) == 3:
        _, source, target = derive(proof[1], context, scope)
        _, left, right = derive(proof[2], context, scope)
        if left != source:
            if left[1] != source:
                fail()
            target = [left[0], target, left[2]]
        return ['=', target, right]
    if tag == 'q' and len(proof) == 4:
        witness, major, base = proof[1:]
        if not term(witness, scope):
            fail()
        closure = derive(major, context, scope)
        if closure[0] != 'Q':
            fail()
        if derive(base, context, scope) != subst(closure[1], [0]):
            fail()
        return subst(closure[1], [witness])
    if tag == 'e' and len(proof) == 4:
        goal, major, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        paired = derive(major, context, scope)
        if paired[0] != 'D':
            fail()
        extended = [shift(a) for a in context] + [['Q', paired[1]]]
        if derive(branch, extended, scope + 1) != shift(goal):
            fail()
        return goal
    if tag == 'i' and len(proof) == 4:
        goal, witness, branch = proof[1:]
        if not formula(goal, scope) or goal[0] != 'D' or not term(witness, scope):
            fail()
        body = shift(goal[1], 1, 2)
        lifted_witness = shift(witness)
        eta = 1 if goal[1][0] == '=' and witness == 0 else 0
        hypothesis = subst(body, [eta, lifted_witness])
        target = subst(body, [successor(1), lifted_witness])
        extended = [shift(a) for a in context] + [hypothesis]
        if derive(branch, extended, scope + 1) != target:
            fail()
        return goal
    fail()
```

```
def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, commit `6bf83d8765614b0af51b35747e07fb057d6b2f42`. https://github.com/VladimirReshetnikov/ProveIt/tree/6bf83d8765614b0af51b35747e07fb057d6b2f42/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] Jeremy Avigad, *Proof Theory and Proof Mining II: Formal Theories of Analysis*, July 2015, slides 4–7. https://www.andrew.cmu.edu/user/avigad/Talks/helsinki/helsinki2.pdf.

[3] ProveIt, inherited *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, at the snapshot in [1].

[4] ProveIt, inherited *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, at the snapshot in [1].

[5] ProveIt, inherited *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, at the snapshot in [1].